cost). As we'll see in section 16.4, that's why it works perfectly with the kind of cost functions we design for supervised learning.

16.3.4 Problems with Gradient Descent

One important thing to notice in listing 16.4 is that we provide a learning rate `alpha`: this is a hyper-parameter of the algorithm, regulating how big are the steps we take: like in the marble analogy, when we don't have a clear view of the track taking large steps can speed us up, but it can also send the marble off-course, similarly in gradient descent large steps can miss minima (figure 16.16.A) or even worse, in some situations they can cause loops or even getting far away from the best solution, in situations like in figure 16.16.B.

Vice versa, when alpha is too small (figure 16.16.C), convergence can be far too slow and the optimization algorithm will never get to a minimum within a reasonable time (or within the maximum number of iterations allowed).

Which values are too big, or too small, for alpha also depends on the context, and in particular on the specific function we are trying to optimize.

If we didn't have the chance to pass this learning parameter, then for cost functions like the one in figure 16.16.B, where the slope of the curve is steep, the optimization would not converge to the minimum (but rather diverge), and at the same time for examples like 16.14.C, convergence would be too slow, or the algorithm could get stuck in local minima.